Requirement Analysis Document

Phase 2: Functional & Non-Functional Specifications

PROJECT PLATFORM:	MediConnect - Premium MERN Doctor Appointment Platform
REFERENCE PATH:	VIP-C2-BOOK-A-DOCTOR
DATE OF EXECUTION:	June 16, 2026
AUTHOR / CREATOR:	Antigravity AI Engineer & pairing USER
DOCUMENT STATUS:	**APPROVED / PRODUCTION-READY**

EXECUTIVE OVERVIEW

This phase-wise document outlines the development lifecycle, system structures, and technical specifications of MediConnect. MediConnect is a clinic-agnostic, hospital-grade scheduling ecosystem built utilizing the MERN stack (MongoDB, Express, React, Node.js). It supports secure JWT access/refresh token structures, comprehensive patient calendars, and simulated multi-modal checkout payment gates.

1. Functional Requirements (FR)

Functional requirements define the core operational features that the MediConnect platform must perform.

Req ID	Feature Domain	Detailed Specification / Criteria
FR-101	User Authentication	Secure login/signup with email confirmation. Implement access and refresh JWT token rotations
FR-103	Calendar Booking Grid	Interactive FullCalendar/Shift grid displaying active available slots for patient selection.
		cash payments.
FR-105	Appointments Tracker	A Patient Appointments panel showing doctor name, fee, date/time, and real-time status.
FR-107	Admin Dashboard Charts	Graphical rendering of monthly registrations, specialty distribution, and platform booking volumes.

2. Non-Functional Requirements (NFR)

Non-functional requirements describe system quality attributes, security measures, and engineering constraints. MediConnect enforces rigorous standards across these dimensions:

• Security & Encryption:
Passwords must be hashed using bcryptjs (rounds=10) prior to storage. API endpoints are protected using Helmet headers and strict CORS configuration. Rate limiting restricts requests to 100 per 15 minutes locally to prevent denial-of-service.

• Performance & Concurrency:
Node.js server executes asynchronous request parsing. MongoDB models are structured to minimize aggregate lookups, storing critical fields like doctor details directly inside the appointment document.

• Responsive Layout & Compatibility:
Frontend React layout is built with Tailwind CSS, supporting mobile, tablet, and desktop display orientations with flexible flex/grid elements.

3. System Use Cases

The diagram below maps user roles to functional endpoints, demonstrating actor permissions within the platform.

FUNCTIONAL USE CASES - SYSTEM VISUALIZATION